

Báo cáo Chuyên sâu: Kiến trúc Đám mây AWS, Mạng lưới Nâng cao và Chiến lược Triển khai Tự động hóa Toàn diện với Terraform

1. Cơ sở Lý luận và Triết lý của Cơ sở hạ tầng dưới dạng Mã (IaC) trên AWS

Trong bối cảnh công nghệ thông tin hiện đại, sự chuyển dịch từ quản lý cơ sở hạ tầng thủ công sang tự động hóa không chỉ là một xu hướng mà là một yêu cầu bắt buộc để đảm bảo tính cạnh tranh, độ tin cậy và khả năng mở rộng của doanh nghiệp. Cơ sở hạ tầng dưới dạng mã (Infrastructure as Code - IaC) đã thay đổi căn bản cách thức các tổ chức thiết kế, triển khai và vận hành tài nguyên công nghệ. Terraform, một công cụ mã nguồn mở do HashiCorp phát triển, đã trở thành tiêu chuẩn thực tế trong lĩnh vực này, đặc biệt là khi kết hợp với nền tảng Amazon Web Services (AWS).¹ Báo cáo này cung cấp một nghiên cứu toàn diện và chuyên sâu về việc sử dụng Terraform để xây dựng kiến trúc mạng AWS, các chiến lược triển khai tiên tiến và quy trình vận hành tự động.

1.1. Sự Cần thiết của Tự động hóa và Tính Bất biến

Mô hình quản lý cơ sở hạ tầng truyền thống dựa vào các thay đổi thủ công (thường được gọi là "ClickOps") trên bảng điều khiển quản trị. Phương pháp này tiềm ẩn nhiều rủi ro, bao gồm sai sót của con người, thiếu tính nhất quán giữa các môi trường (configuration drift) và khó khăn trong việc kiểm toán. IaC giải quyết các vấn đề này bằng cách cho phép các kỹ sư định nghĩa toàn bộ cơ sở hạ tầng thông qua các tệp cấu hình có thể đọc được bởi máy và con người.²

Một khái niệm quan trọng trong IaC là "Cơ sở hạ tầng Bất biến" (Immutable Infrastructure). Thay vì cập nhật, vá lỗi hoặc sửa đổi cấu hình trực tiếp trên các máy chủ đang chạy (mutable), mô hình bất biến đề xuất việc thay thế hoàn toàn các tài nguyên cũ bằng các tài nguyên mới được xây dựng từ đầu mỗi khi có thay đổi.³ Terraform hỗ trợ mạnh mẽ mô hình này thông qua quy trình plan và apply, cho phép phá hủy và tạo mới tài nguyên một cách có kiểm soát. Việc này đảm bảo rằng trạng thái của hệ thống luôn đồng nhất với mã nguồn đã được kiểm thử, loại bỏ các "bóng ma" cấu hình tích tụ theo thời gian.⁵

1.2. Terraform Provider cho AWS: Cơ chế và Quản lý Phiên bản

Terraform tương tác với AWS thông qua một plugin gọi là AWS Provider. Provider này chịu trách nhiệm dịch các cú pháp HCL (HashiCorp Configuration Language) thành các lệnh gọi

API của AWS để tạo, đọc, cập nhật và xóa tài nguyên.⁶ Việc quản lý phiên bản của AWS Provider là một yếu tố then chốt để đảm bảo sự ổn định của hệ thống.

Các thực tiễn tốt nhất khuyến nghị việc "ghim" (pinning) phiên bản của Provider trong khối `required_providers`. Việc cho phép Terraform tự động tải xuống phiên bản mới nhất (ví dụ: sử dụng ký tự đại diện `~>`) có thể dẫn đến các lỗi không mong muốn nếu phiên bản mới chứa các thay đổi phá vỡ tương thích (breaking changes).⁷ Các kỹ sư nên thiết lập quy trình tự động trong CI/CD để kiểm tra và cảnh báo nếu phiên bản Provider không được ghim cụ thể, sử dụng các công cụ như TFLint để thực thi quy tắc này. Hơn nữa, việc theo dõi các bản phát hành mới của Provider và thử nghiệm nâng cấp trong môi trường phi sản xuất (non-production) trước khi áp dụng cho môi trường sản xuất là quy trình bắt buộc để giảm thiểu rủi ro.

1.3. Quản lý Trạng thái (State Management): Nền tảng của Sự Cộng tác

Tập trạng thái (`terraform.tfstate`) là "bộ não" của Terraform, lưu trữ ánh xạ giữa các tài nguyên trong mã cấu hình và các đối tượng thực tế trên AWS. Một trong những sai lầm phổ biến nhất và nguy hiểm nhất là lưu trữ tập trạng thái này cục bộ trên máy tính cá nhân hoặc cam kết nó vào hệ thống quản lý mã nguồn (Git).⁷

Kiến trúc Backend Chuẩn mực:

Để hỗ trợ làm việc nhóm và đảm bảo an toàn dữ liệu, mô hình tiêu chuẩn được khuyến nghị là sử dụng Amazon S3 làm backend lưu trữ trạng thái và Amazon DynamoDB để quản lý khóa (state locking).⁹

- **Lưu trữ trên S3:** S3 cung cấp độ bền dữ liệu vượt trội (99.999999999%) và tính sẵn sàng cao. Cấu hình bucket S3 cho Terraform state cần tuân thủ nghiêm ngặt các yêu cầu bảo mật: kích hoạt mã hóa phía máy chủ (Server-Side Encryption) để bảo vệ dữ liệu nhạy cảm (như mật khẩu database, access keys) và kích hoạt tính năng Lập phiên bản (Versioning).⁹ Việc lập phiên bản cho phép khôi phục lại trạng thái cũ trong trường hợp tập state bị hỏng hoặc bị ghi đè do lỗi con người, đóng vai trò như một cơ chế sao lưu tức thời.⁹
- **Cơ chế Khóa với DynamoDB:** Trong môi trường cộng tác, rủi ro "Race Condition" xảy ra khi hai kỹ sư cùng thực hiện lệnh `terraform apply` đồng thời. Điều này có thể dẫn đến hỏng tập state và mất mát dữ liệu hạ tầng. DynamoDB giải quyết vấn đề này bằng cách lưu trữ một khóa (lock) với Partition Key là `LockID`. Khi Terraform bắt đầu một tác vụ ghi, nó sẽ tạo một bản ghi trong DynamoDB để khóa state; các tác vụ khác sẽ bị từ chối cho đến khi khóa được giải phóng.⁹

Phân tách State theo Môi trường:

Một nguyên tắc vàng trong quản lý state là sự cô lập. Không bao giờ nên sử dụng chung một state file cho các môi trường khác nhau (Dev, Staging, Prod). Việc này đảm bảo rằng một lỗi thao tác trên môi trường Dev không bao giờ có thể vô tình phá hủy tài nguyên của Prod.⁹ Terraform Workspaces cung cấp một cách nhẹ nhàng để quản lý nhiều state cho cùng một mã nguồn, nhưng đối với các hệ thống lớn, việc tách biệt hoàn toàn ở cấp độ thư mục (Directory

Separation) hoặc thậm chí là tài khoản AWS (Multi-account strategy) được ưu tiên hơn để tối đa hóa sự an toàn.¹¹

2. Thiết kế Kiến trúc Mạng AWS Nâng cao (Advanced Networking)

Mạng lưới (Networking) là xương sống của mọi ứng dụng đám mây. Một kiến trúc Virtual Private Cloud (VPC) được thiết kế tốt phải đảm bảo tính bảo mật, khả năng mở rộng và hiệu quả chi phí. Terraform cung cấp các công cụ mạnh mẽ để định nghĩa và quản lý cấu trúc mạng phức tạp này một cách nhất quán.

2.1. Quy hoạch VPC và Chiến lược Phân đoạn (Segmentation)

Bước đầu tiên trong việc xây dựng mạng lưới là định nghĩa tài nguyên `aws_vpc`. Việc lựa chọn dải địa chỉ IP (CIDR block) cần được cân nhắc kỹ lưỡng để tránh xung đột với các mạng nội bộ (on-premise) hoặc các VPC khác trong tương lai nếu cần thiết lập kết nối Peering hoặc VPN.¹³ Các thuộc tính như `enable_dns_support` và `enable_dns_hostnames` nên được thiết lập là `true` để hỗ trợ phân giải tên miền nội bộ, điều kiện tiên quyết quan trọng cho các dịch vụ như Application Load Balancer (ALB), Amazon EFS, và các hệ thống điều phối container như EKS hoặc ECS.¹³

Phân đoạn Mạng (Subnetting):

Phân đoạn mạng là kỹ thuật chia nhỏ VPC thành các mạng con (subnets) để kiểm soát luồng giao thông và tăng cường bảo mật. Một mô hình phân đoạn tiêu chuẩn cho ứng dụng đa tầng (multi-tier application) thường bao gồm 15:

- **Public Subnets:** Các subnet này có đường định tuyến trực tiếp ra Internet thông qua Internet Gateway (IGW). Chúng chứa các tài nguyên cần giao tiếp hai chiều với Internet công cộng, chẳng hạn như Public Load Balancers, Bastion Hosts, hoặc NAT Gateways.¹⁴
- **Private Subnets (Application Tier):** Đây là nơi đặt các máy chủ ứng dụng, backend services hoặc worker nodes của Kubernetes. Các tài nguyên trong subnet này không có địa chỉ IP công cộng và không thể truy cập trực tiếp từ Internet. Kết nối ra ngoài (outbound) chỉ được cho phép thông qua NAT Gateway.¹⁵
- **Data/Database Subnets:** Lớp mạng con biệt lập nhất, dành riêng cho các hệ thống lưu trữ dữ liệu như RDS, ElastiCache, hoặc Redshift. Các subnet này thường không có đường ra Internet (thậm chí không qua NAT) và được bảo vệ chặt chẽ bởi Network ACLs, chỉ cho phép kết nối từ Application Tier.¹⁷

Việc sử dụng Terraform để triển khai mô hình này cho phép tạo ra các mẫu tái sử dụng. Bằng cách sử dụng các biến (variables) cho danh sách CIDR và Availability Zones (AZ), kỹ sư có thể triển khai cùng một cấu trúc mạng trên nhiều khu vực (Regions) khác nhau mà không cần viết lại mã, đảm bảo tính nhất quán tuyệt đối.¹³

2.2. Cơ chế Định tuyến và Kết nối Internet

Hệ thống định tuyến (Routing) trong VPC quyết định cách thức các gói tin di chuyển giữa các subnet và ra thế giới bên ngoài.

- **Internet Gateway (IGW):** Tài nguyên `aws_internet_gateway` là cổng kết nối duy nhất cho phép VPC giao tiếp với Internet. Trong Terraform, IGW được gắn vào VPC và một Route Table (thường gọi là Public Route Table) sẽ được cấu hình với tuyến đường mặc định `0.0.0.0/0` trở về IGW này.¹⁶
- **NAT Gateway:** Đối với các Private Subnets, nhu cầu truy cập Internet để tải cập nhật phần mềm hoặc gọi API bên ngoài là phổ biến. NAT Gateway (Network Address Translation) cho phép các instance trong private subnet khởi tạo kết nối ra ngoài nhưng chặn các kết nối từ bên ngoài đi vào.¹⁶ Để đảm bảo tính sẵn sàng cao (High Availability - HA), kiến trúc tốt nhất là triển khai một NAT Gateway riêng biệt cho mỗi Availability Zone. Nếu một AZ gặp sự cố, các instance trong AZ khác vẫn duy trì được kết nối.¹⁹ Tuy nhiên, điều này làm tăng chi phí đáng kể.

Tối ưu hóa Chi phí: NAT Gateway vs. NAT Instance:

Managed NAT Gateway của AWS tính phí dựa trên thời gian hoạt động và lưu lượng dữ liệu xử lý. Trong các môi trường phát triển (Dev) hoặc kiểm thử (Staging) nơi tính sẵn sàng cao không phải là ưu tiên hàng đầu, chi phí này có thể trở thành gánh nặng. Một giải pháp thay thế hiệu quả về chi phí là sử dụng NAT Instance (một EC2 instance được cấu hình làm NAT). Terraform cho phép sử dụng logic điều kiện (conditional logic) với `count` hoặc `for_each` để triển khai NAT Gateway trong môi trường Production và NAT Instance trong môi trường Non-Prod, giúp tiết kiệm tới 90% chi phí mạng trong các môi trường này.²¹

2.3. VPC Endpoints: Tối ưu Hóa và Bảo mật Kết nối Dịch vụ

Trong kiến trúc mạng hiện đại, việc truy cập các dịch vụ AWS (như S3, DynamoDB) thông qua Internet công cộng hoặc NAT Gateway là không tối ưu cả về bảo mật lẫn chi phí. VPC Endpoints cung cấp kết nối riêng tư (private connectivity) tới các dịch vụ này.²³

Loại Endpoint	Chi tiết Kỹ thuật	Ưu điểm & Ứng dụng	Lưu ý về Terraform
Gateway Endpoint	Sử dụng entry trong Route Table để định tuyến traffic. Chỉ hỗ trợ S3 và DynamoDB.	Miễn phí hoàn toàn (không phí giờ, không phí data). Giảm tải cho NAT Gateway, tiết kiệm chi phí xử lý dữ liệu. ²⁴	Cấu hình qua <code>aws_vpc_endpoint</code> với <code>vpc_endpoint_type = "Gateway"</code> . Cần liên kết với các Route Table cụ thể. ²⁵

Interface Endpoint	Sử dụng Elastic Network Interface (ENI) với Private IP trong subnet. Hỗ trợ hầu hết dịch vụ AWS (CloudWatch, SNS, EC2 API...).	Cho phép kết nối từ on-premise qua VPN/Direct Connect. Hỗ trợ Security Groups để kiểm soát truy cập chi tiết. ²⁶	Có phí theo giờ và phí dữ liệu. Cấu hình <code>private_dns_enabled = true</code> để ứng dụng tự động dùng endpoint mà không cần sửa code. ²⁵
---------------------------	--	---	---

Việc tích hợp Gateway Endpoints cho S3 và DynamoDB vào mọi thiết kế VPC là một "best practice" được khuyến nghị mạnh mẽ để tối ưu hóa chi phí và hiệu năng.²¹

3. Hệ thống Phòng thủ Mạng: Security Groups và Network ACLs

Bảo mật mạng trên AWS tuân theo nguyên tắc "Phòng thủ theo chiều sâu" (Defense in Depth), sử dụng nhiều lớp kiểm soát để bảo vệ tài nguyên. Hai công cụ chính là Security Groups (SG) và Network Access Control Lists (NACLs).

3.1. So sánh Chiến lược và Cơ chế Hoạt động

Hiểu rõ sự khác biệt giữa SG và NACL là cốt lõi của thiết kế bảo mật:

- **Security Groups (SG):** Hoạt động ở cấp độ Instance (cụ thể là ENI). SG là tường lửa **có trạng thái** (stateful). Điều này có nghĩa là nếu một yêu cầu gửi đi (outbound) được cho phép, phản hồi (inbound) tương ứng sẽ tự động được chấp nhận, bất kể quy tắc inbound hiện tại.²⁷ SG là lớp bảo vệ chính và linh hoạt nhất cho ứng dụng.
- **Network ACLs (NACLs):** Hoạt động ở cấp độ Subnet. NACL là tường lửa **không trạng thái** (stateless), nghĩa là các quy tắc inbound và outbound phải được định nghĩa riêng biệt. NACL xử lý lưu lượng dựa trên thứ tự ưu tiên của quy tắc (rule number) và hỗ trợ hành động DENY (từ chối), điều mà SG không có.²⁷

3.2. Thực tiễn Tốt nhất trong Cấu hình Terraform

Khi triển khai các quy tắc bảo mật này bằng Terraform, cần tuân thủ các nguyên tắc sau để đảm bảo khả năng quản lý và bảo mật:

1. **SG Referencing:** Thay vì sử dụng các dải IP (CIDR blocks) cứng nhắc trong các quy tắc SG, hãy tham chiếu đến ID của các Security Group khác. Ví dụ: SG của Database chỉ cho phép inbound từ SG của App Server. Điều này tạo ra một chuỗi tin cậy logic, đảm bảo rằng chỉ các tài nguyên thuộc lớp ứng dụng mới có thể truy cập cơ sở dữ liệu, bất kể địa chỉ IP của chúng thay đổi như thế nào (do Auto Scaling).³⁰
2. **Tách biệt Quy tắc:** Sử dụng tài nguyên `aws_security_group_rule` riêng biệt thay vì định nghĩa các khối ingress/egress nội tuyến trong tài nguyên `aws_security_group`. Cách tiếp

cận này giúp tránh xung đột vòng lặp (circular dependencies) và cho phép quản lý quy tắc linh hoạt hơn trong các module phức tạp.³¹

3. **NACLs như một Lớp Bảo vệ Thông:** Sử dụng NACL để thiết lập các rào chắn rộng, ví dụ như chặn các dải IP độc hại đã biết hoặc cô lập toàn bộ subnet khi bị tấn công DDoS. Không nên sử dụng NACL để quản lý quyền truy cập chi tiết của từng instance vì tính phức tạp của việc quản lý ephemeral ports trong stateless firewall.³⁰
4. **Nguyên tắc Đặc quyền Tối thiểu:** Chỉ mở các cổng và giao thức thực sự cần thiết. Sử dụng terraform-docs hoặc các chú thích trong mã để giải thích lý do tại sao một cổng cụ thể được mở, hỗ trợ quá trình kiểm toán sau này.³³

4. Kiến trúc Module và Tổ chức Mã nguồn Terraform

Khi quy mô cơ sở hạ tầng phát triển, việc quản lý hàng nghìn dòng mã Terraform trong một tệp main.tf duy nhất trở nên bất khả thi. Mô-đun hóa (Modularity) là chìa khóa để duy trì sự gọn gàng, khả năng tái sử dụng và khả năng mở rộng của dự án.

4.1. Thiết kế Module Chuẩn mực

Một cấu trúc module hiệu quả thường được chia thành hai loại:

- **Module Cơ sở (Resource/Foundational Modules):** Bao gói các tài nguyên đơn lẻ hoặc nhóm tài nguyên liên quan chặt chẽ (ví dụ: module vpc, module security-group, module iam-role). Các module này nên tập trung vào tính linh hoạt và khả năng cấu hình cao.³¹
- **Module Ứng dụng/Kiến trúc (Composition/Application Modules):** Kết hợp các module cơ sở để tạo thành một giải pháp trọn vẹn (ví dụ: module web-service bao gồm EC2, SG, ALB và IAM). Loại module này định nghĩa các mẫu kiến trúc chuẩn của tổ chức (architectural patterns).⁶

Nguyên tắc "Không bọc mỏng" (Avoid Thin Wrappers): Tránh tạo các module chỉ đơn thuần bọc lại một tài nguyên Terraform duy nhất mà không thêm giá trị logic nào. Điều này chỉ làm tăng độ phức tạp mà không mang lại lợi ích. Module nên cung cấp sự trừu tượng hóa có ý nghĩa, ví dụ như thiết lập các giá trị mặc định an toàn hoặc kết hợp các tài nguyên phụ thuộc.³¹

4.2. Quản lý Môi trường và Sự Phân tách

Việc tổ chức mã nguồn để hỗ trợ nhiều môi trường (Dev, Staging, Prod) là một thách thức lớn. Có hai mô hình chính được áp dụng:

1. **Sử dụng Terraform Workspaces:** Cho phép sử dụng cùng một bộ mã cấu hình nhưng duy trì các state file riêng biệt cho từng workspace. Mô hình này phù hợp khi các môi trường giống hệt nhau về mặt kiến trúc và chỉ khác nhau về quy mô (ví dụ: số lượng instance).¹¹ Tuy nhiên, nó có thể gây nhầm lẫn vì không hiển thị rõ ràng sự khác biệt cấu hình trong cấu trúc thư mục.
2. **Phân tách Thư mục (Directory Structure):** Tạo các thư mục riêng biệt cho mỗi môi trường.

trường (ví dụ: envs/dev, envs/prod). Mỗi thư mục chứa một tệp main.tf gọi đến các module chung nhưng truyền vào các biến số khác nhau. Đây là phương pháp được khuyến nghị cho các dự án lớn vì nó cung cấp sự rõ ràng, cho phép tùy biến phiên bản module cho từng môi trường (ví dụ: Prod dùng module v1.0, Dev dùng module v1.1-beta) và giảm thiểu rủi ro thao tác nhầm lẫn.¹²

4.3. Kết nối Lỏng lẻo với Remote State Data Sources

Trong kiến trúc microservices hoặc các hệ thống lớn, việc tách biệt lớp Mạng (Networking Layer) và lớp Ứng dụng (Application Layer) thành các dự án Terraform riêng biệt là rất cần thiết. Lớp Mạng (VPC, Subnets) thường ít thay đổi, trong khi lớp Ứng dụng thay đổi liên tục. Để kết nối chúng, Terraform cung cấp terraform_remote_state data source.

Kỹ thuật này cho phép dự án Ứng dụng "đọc" các thông tin đầu ra (outputs) như vpc_id hay private_subnet_ids từ state file của dự án Mạng (lưu trên S3) mà không cần can thiệp vào nó. Điều này tạo ra sự liên kết lỏng lẻo (loose coupling), cho phép các nhóm mạng và ứng dụng làm việc độc lập nhưng vẫn đảm bảo sự tích hợp hệ thống.¹²

5. Các Chiến lược Triển khai Ứng dụng Tiên tiến (Deployment Patterns)

Mục tiêu tối thượng của việc triển khai là đưa mã mới vào sử dụng với thời gian chết (downtime) bằng không và rủi ro thấp nhất. Terraform, kết hợp với các dịch vụ AWS, đóng vai trò là nhạc trưởng điều phối các chiến lược triển khai phức tạp này.

5.1. Blue/Green Deployment: An toàn Tuyệt đối

Blue/Green Deployment là chiến lược duy trì hai môi trường sản xuất song song: "Blue" (phiên bản hiện tại, đang phục vụ người dùng) và "Green" (phiên bản mới, đang chờ kiểm thử).

Cơ chế hoạt động với Terraform và ALB:

1. **Khởi tạo:** Môi trường Blue đang chạy và nhận 100% traffic từ Application Load Balancer (ALB).
2. **Triển khai Green:** Terraform tạo một tập hợp tài nguyên mới (Auto Scaling Group, EC2 instances) cho môi trường Green. Các instance này được đăng ký vào một Target Group mới (Green Target Group).³⁵
3. **Kiểm thử:** Traffic nội bộ hoặc traffic trên một port thử nghiệm được định tuyến vào Green Target Group để kiểm tra tính ổn định.
4. **Chuyển đổi (Cutover):** Thông qua việc cập nhật aws_lb_listener rule trong Terraform, trọng số lưu lượng (traffic weight) được điều chỉnh. Traffic được chuyển từ Blue sang Green. Nếu sử dụng biến traffic_distribution, kỹ sư có thể kiểm soát chính xác tỷ lệ này (ví dụ: chuyển ngay lập tức 100% hoặc chuyển từ từ).³⁵
5. **Dọn dẹp:** Sau khi Green hoạt động ổn định, Terraform hủy bỏ tài nguyên của môi trường Blue (hoặc giữ lại làm dự phòng).

RDS Blue/Green Deployments:

Thách thức lớn nhất của Blue/Green thường nằm ở cơ sở dữ liệu. AWS RDS hiện hỗ trợ "Fully Managed Blue/Green Deployments". Terraform hỗ trợ tính năng này thông qua khối `blue_green_update` trong tài nguyên `aws_db_instance`. Khi được kích hoạt, AWS tự động tạo một môi trường staging cho DB, đồng bộ dữ liệu, và cho phép nâng cấp phiên bản engine hoặc thay đổi schema trên môi trường Green mà không ảnh hưởng đến Blue. Quá trình switchover diễn ra trong chưa đầy một phút mà không mất mát dữ liệu.³⁷

5.2. Canary Deployment: Giảm thiểu Phạm vi Ảnh hưởng

Canary Deployment là một biến thể tinh vi hơn, nơi phiên bản mới được phát hành cho một nhóm nhỏ người dùng trước. Terraform hỗ trợ mô hình này thông qua tính năng `weighted routing` của ALB hoặc Route 53.

Bằng cách thiết lập cấu hình `forward action` trong `aws_lb_listener` với các khối `target_group` có trọng số khác nhau (ví dụ: 90% cho Blue, 10% cho Green), Terraform cho phép kỹ sư quan sát hiệu năng của phiên bản mới trên thực tế với rủi ro thấp. Nếu các chỉ số giám sát (metrics) ổn định, trọng số của Green sẽ được tăng dần cho đến khi đạt 100%.³⁵

5.3. Rolling Updates và Cơ sở hạ tầng Bất biến

Trong mô hình Rolling Update, các instance cũ dần dần được thay thế bởi các instance mới. Terraform quản lý quy trình này thông qua `aws_autoscaling_group` và `launch_template`.

Cơ sở hạ tầng Bất biến (Immutable Infrastructure):

Thay vì SSH vào từng server để chạy `apt-get update` (mutable), quy trình hiện đại sử dụng Packer để đóng gói toàn bộ hệ điều hành, thư viện và mã ứng dụng vào một Amazon Machine Image (AMI) tĩnh.⁴¹

Quy trình tích hợp (Pipeline) điển hình:

1. **Packer Build:** Jenkins hoặc GitHub Actions chạy Packer để tạo AMI mới. Packer xuất ra AMI ID.
2. **Terraform Update:** Terraform nhận AMI ID mới (thông qua biến đầu vào hoặc data source `aws_ami` lọc theo tag mới nhất).
3. **Instance Refresh:** Terraform cập nhật Launch Template với AMI ID mới. Tính năng `instance_refresh` của ASG được kích hoạt, tự động thay thế các instance cũ theo lô (batch) dựa trên `min_healthy_percentage` cấu hình sẵn, đảm bảo dịch vụ không bao giờ bị gián đoạn.⁴³

Sự kết hợp giữa Packer và Terraform hiện thực hóa triết lý bất biến: mỗi thay đổi đều tạo ra một máy chủ hoàn toàn mới, loại bỏ cấu hình sai lệch và đảm bảo môi trường sản xuất luôn giống hệt môi trường kiểm thử.⁵

6. Tự động hóa Quy trình CI/CD và Vận hành Xuất sắc

Để đạt được tốc độ và độ tin cậy cao, việc chạy Terraform thủ công từ máy trạm của kỹ sư phải được loại bỏ hoàn toàn. Thay vào đó, một đường ống CI/CD (Continuous

Integration/Continuous Deployment) tự động là bắt buộc.

6.1. Kiến trúc Pipeline Terraform với GitHub Actions

Một quy trình CI/CD tiêu chuẩn cho Terraform, sử dụng GitHub Actions, bao gồm các giai đoạn kiểm soát chặt chẽ ⁴⁵:

1. **Kiểm tra Chất lượng (Lint & Validate)**: Ngay khi có Pull Request (PR), pipeline chạy terraform fmt để kiểm tra định dạng và terraform validate để kiểm tra tính hợp lệ của cấu hình.
2. **Quét Bảo mật (Security Scanning)**: Tích hợp các công cụ phân tích tĩnh (Static Analysis Security Testing - SAST) như **Checkov**, **Tfsec** hoặc **Terrascan**. Các công cụ này quét mã Terraform để phát hiện các lỗ hổng bảo mật tiềm ẩn (như S3 bucket công khai, thiếu mã hóa EBS, Security Group quá mở) và chặn PR nếu phát hiện vi phạm nghiêm trọng.⁶
3. **Lập Kế hoạch (Plan)**: Pipeline thực thi terraform plan và xuất kết quả ra một tệp. Quan trọng hơn, kết quả plan nên được tự động comment vào PR để người review có thể thấy rõ các thay đổi dự kiến (những tài nguyên nào sẽ bị thêm, sửa, xóa).⁴⁷
4. **Phê duyệt và Áp dụng (Apply)**: Chỉ khi PR được merge vào nhánh chính (main branch), bước terraform apply mới được thực thi. Quá trình này nên sử dụng OpenID Connect (OIDC) để xác thực với AWS thay vì lưu trữ Access Keys lâu dài, tuân thủ nguyên tắc bảo mật hiện đại.⁴⁶

6.2. Phát hiện và Xử lý Trôi dạt (Drift Detection)

"Drift" là hiện tượng cơ sở hạ tầng thực tế trên AWS bị thay đổi thủ công (ClickOps) và không còn khớp với mã Terraform. Điều này làm suy yếu giá trị của IaC.

Để đối phó, các tổ chức nên thiết lập một Scheduled Workflow (ví dụ: chạy hàng đêm) trong GitHub Actions. Workflow này thực hiện lệnh terraform plan -detailed-exitcode. Nếu lệnh trả về mã thoát là 2 (có thay đổi), nghĩa là Drift đã xảy ra. Hệ thống sau đó tự động tạo một GitHub Issue, gửi cảnh báo Slack hoặc Email cho đội ngũ vận hành để điều tra và khắc phục (hoặc cập nhật lại mã Terraform, hoặc hoàn tác thay đổi thủ công).⁴⁸

7. Kết luận

Việc xây dựng và quản lý cơ sở hạ tầng trên AWS bằng Terraform là một hành trình đòi hỏi sự kết hợp chặt chẽ giữa tư duy kiến trúc, chiến lược bảo mật và quy trình tự động hóa. Từ việc thiết kế mạng VPC phân đoạn, áp dụng các lớp bảo mật đa tầng với SG và NACL, đến việc triển khai các chiến lược Blue/Green và quản lý state an toàn, mỗi quyết định đều ảnh hưởng trực tiếp đến độ ổn định và khả năng mở rộng của hệ thống. Bằng cách tuân thủ các thực tiễn tốt nhất về IaC, đóng gói bất biến với Packer và tự động hóa toàn diện qua CI/CD, các tổ chức có thể biến cơ sở hạ tầng từ một gánh nặng quản lý thành một lợi thế cạnh tranh, cho phép đổi mới nhanh chóng và an toàn.

Bảng Tổng hợp: So sánh Chiến lược Triển khai trên AWS với Terraform

Chiến lược	Cơ chế Terraform	Ưu điểm	Nhược điểm
Blue/Green	aws_lb_listener chuyển trọng số traffic giữa 2 Target Groups.	Zero downtime, rollback tức thì, môi trường sạch.	Chi phí cao (nhân đôi tài nguyên), phức tạp trong quản lý DB.
Canary	weighted_routing trên ALB hoặc Route 53.	Giảm thiểu rủi ro (blast radius nhỏ), kiểm thử trên người dùng thật.	Triển khai chậm hơn, cần giám sát chặt chẽ metrics.
Rolling Update	aws_autoscaling_group với instance_refresh.	Tiết kiệm chi phí (không nhân đôi), tự động hóa cao.	Rollback khó khăn và chậm hơn, có trạng thái hỗn hợp trong quá trình update.
Recreate	create_before_destroy = false.	Đơn giản, trạng thái sạch hoàn toàn.	Gây downtime, không phù hợp cho hệ thống production quan trọng.
Nguồn trích dẫn	35	40	43

Nguồn trích dẫn

1. Terraform - HashiCorp Developer, truy cập vào tháng 12 18, 2025, <https://developer.hashicorp.com/terraform>
2. What is Terraform | Terraform - HashiCorp Developer, truy cập vào tháng 12 18, 2025, <https://developer.hashicorp.com/terraform/intro>
3. Terraform for AWS in Practice (Part 3): CI/CD, Security, and Scaling Strategies, truy cập vào tháng 12 18, 2025, <https://www.cloudoptimo.com/blog/terraform-for-aws-in-practice-part-3-ci-cd-security-and-scaling-strategies/>
4. REL08-BP04 Deploy using immutable infrastructure - AWS Well-Architected Framework, truy cập vào tháng 12 18, 2025,

- https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_tracking_change_management_immutable_infrastructure.html
5. (Im)mutable Infrastructure and Terraform | by Migara Ekanayake - Medium, truy cập vào tháng 12 18, 2025, <https://medium.com/@migara/im-mutable-infrastructure-and-terraform-f50e78de0912>
 6. AWS Prescriptive Guidance - Best practices for using the Terraform AWS Provider - AWS Documentation, truy cập vào tháng 12 18, 2025, <https://docs.aws.amazon.com/pdfs/prescriptive-guidance/latest/terraform-aws-provider-best-practices/terraform-aws-provider-best-practices.pdf>
 7. The Most Common Terraform Mistakes (and How to Avoid Them) | by Piyush Chauhan, truy cập vào tháng 12 18, 2025, <https://medium.com/@piyushkumar3318/the-most-common-terraform-mistakes-and-how-to-avoid-them-c374f47323af>
 8. Building a Production Terraform Backend on AWS: S3, DynamoDB & Best Practices | by Rahul Shelke | Dec, 2025 | Medium, truy cập vào tháng 12 18, 2025, <https://medium.com/@rahulshelke3399/building-a-production-terraform-backend-on-aws-s3-dynamodb-best-practices-2c345ef2873b>
 9. How to Manage Terraform S3 Backend - Best Practices - Spacelift, truy cập vào tháng 12 18, 2025, <https://spacelift.io/blog/terraform-s3-backend>
 10. Best practices for using the Terraform AWS Provider - AWS ..., truy cập vào tháng 12 18, 2025, <https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/introduction.html>
 11. Best practices when using Terraform? Is there any official guide to it? - Stack Overflow, truy cập vào tháng 12 18, 2025, <https://stackoverflow.com/questions/33157516/best-practices-when-using-terraform-is-there-any-official-guide-to-it>
 12. How I Split My Terraform Infrastructure into Independent Layers | by Liantsoa R - Medium, truy cập vào tháng 12 18, 2025, <https://medium.com/@rmliantsoa/how-i-split-my-terraform-infrastructure-into-independent-layers-9cfa014a7fce>
 13. How I Built a Production-Ready AWS VPC Using Terraform | by ..., truy cập vào tháng 12 18, 2025, <https://aws.plainenglish.io/how-i-built-a-production-ready-aws-vpc-using-terraform-e4cb780078d4>
 14. Terraform VPC Mini Project: A Complete Step-by-Step Guide to Building a Secure AWS Network, truy cập vào tháng 12 18, 2025, <https://www.youtube.com/watch?v=qPmaP-nDWg8&vl=en>
 15. Secure by default AWS networking with Terraform — Subnet Segmentation | by Andrew Larsen | Dec, 2025 | Medium, truy cập vào tháng 12 18, 2025, <https://medium.com/@andrew-larse514/aws-vpc-network-security-segmenting-your-network-03ba1ae38158>
 16. Build a Custom VPC with Bastion Host, NAT Gateway, and Web Server using Terraform, truy cập vào tháng 12 18, 2025,

- <https://hbayraktar.medium.com/build-a-custom-vpc-with-bastion-host-nat-gateway-and-web-server-using-terraform-48afc864f916>
17. Terraform Your VPC: Create Public, Private & Database Subnets in Two AZs - Medium, truy cập vào tháng 12 18, 2025, <https://medium.com/@anuragabcr/terraform-your-vpc-create-public-private-database-subnets-in-two-azs-f7359ff8cd2e>
 18. How to Build AWS VPC & Subnets using Terraform - Step by Step - Spacelift, truy cập vào tháng 12 18, 2025, <https://spacelift.io/blog/terraform-aws-vpc>
 19. aws_nat_gateway | Resources | hashicorp/aws - Terraform Registry, truy cập vào tháng 12 18, 2025, https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/nat_gateway
 20. Placing EC2 Webserver Instances in a Private Subnet with Internet Access via NAT Gateway using Terraform - DEV Community, truy cập vào tháng 12 18, 2025, <https://dev.to/chinmay13/placing-ec2-webserver-instances-in-a-private-subnet-with-internet-access-via-nat-gateway-using-terraform-167n>
 21. AWS Cloud Cost Optimization in NAT Gateway | CloudKeeper, truy cập vào tháng 12 18, 2025, <https://www.cloudkeeper.com/insights/blog/aws-cloud-cost-optimization-nat-gateway>
 22. From AWS NAT Gateway to NAT Instance: A Cost-Optimized Networking Strategy, truy cập vào tháng 12 18, 2025, <https://blogs.halodoc.io/from-aws-nat-gateway-to-nat-instance-a-cost-optimized-networking-strategy/>
 23. Gateway endpoints for Amazon DynamoDB - Amazon Virtual Private Cloud, truy cập vào tháng 12 18, 2025, <https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-ddb.html>
 24. AWS VPC Gateway Endpoints - The Most Underrated Cost Savers - TechOps Examples, truy cập vào tháng 12 18, 2025, <https://www.techopsexamples.com/p/aws-vpc-gateway-endpoints-the-most-underrated-cost-savers>
 25. aws_vpc_endpoint | Resources | hashicorp/aws - Terraform Registry, truy cập vào tháng 12 18, 2025, https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpc_endpoint.html
 26. Creating and Linking VPC Endpoint of Type Interface using Terraform and AWS management console - DEV Community, truy cập vào tháng 12 18, 2025, <https://dev.to/sepiyush/creating-and-linking-vpc-endpoint-of-type-interface-using-terraform-and-aws-management-console-2oo>
 27. AWS Security Group: Best Practices & Instructions - CoreStack, truy cập vào tháng 12 18, 2025, <https://www.corestack.io/aws-security-best-practices/aws-security-group-best-practices/>
 28. AWS Security group vs Network ACLs - Stack Overflow, truy cập vào tháng 12 18, 2025,

- <https://stackoverflow.com/questions/63872895/aws-security-group-vs-network-acls>
29. AWS Security Groups and Network Access Control Lists (NACLs) | by Subham Pradhan, truy cập vào tháng 12 18, 2025,
<https://medium.com/@subhampradhan966/aws-security-groups-and-network-access-control-lists-nacls-4add51916150>
 30. AWS Network Security Showdown: Network ACLs vs. Security Groups Demystified - DEV Community, truy cập vào tháng 12 18, 2025,
<https://dev.to/pkkolla/aws-network-security-showdown-network-acls-vs-security-groups-demystified-40k2>
 31. Best practices for code base structure and organization - AWS Prescriptive Guidance, truy cập vào tháng 12 18, 2025,
<https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/structure.html>
 32. Security groups vs NACLs : r/aws - Reddit, truy cập vào tháng 12 18, 2025,
https://www.reddit.com/r/aws/comments/1amt765/security_groups_vs_nacls/
 33. Security best practices - AWS Prescriptive Guidance, truy cập vào tháng 12 18, 2025,
<https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/security.html>
 34. The terraform_remote_state Data Source | Terraform - HashiCorp Developer, truy cập vào tháng 12 18, 2025,
<https://developer.hashicorp.com/terraform/language/state/remote-state-data>
 35. Use Application Load Balancers for blue-green and canary ..., truy cập vào tháng 12 18, 2025,
<https://developer.hashicorp.com/terraform/tutorials/aws/blue-green-canary-tests-deployments>
 36. Blue/green Deployment On AWS: A Practical Guide |, truy cập vào tháng 12 18, 2025,
<https://octopus.com/devops/software-deployments/blue-green-deployment-aws/>
 37. aws_db_instance | Resources | hashicorp/aws - Terraform Registry, truy cập vào tháng 12 18, 2025,
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/db_instance
 38. AWS RDS Blue/Green Upgrade while Managing Infrastructure in Terraform - Medium, truy cập vào tháng 12 18, 2025,
<https://medium.com/@mdotsalman/aws-rds-blue-green-upgrade-while-managing-infrastructure-in-terraform-362f6596eaf5>
 39. How to Trigger Blue-Green Deployment for AWS RDS from Terraform? - Stack Overflow, truy cập vào tháng 12 18, 2025,
<https://stackoverflow.com/questions/78040395/how-to-trigger-blue-green-deployment-for-aws-rds-from-terraform>
 40. New Terraform Tutorial: Use Application Load Balancers for Blue-Green and Canary Deployments - HashiCorp, truy cập vào tháng 12 18, 2025,

<https://www.hashicorp.com/en/blog/terraform-tutorial-application-load-balancers-for-blue-green-and-canary-deployments>

41. Provision infrastructure with Packer | Terraform - HashiCorp Developer, truy cập vào tháng 12 18, 2025,
<https://developer.hashicorp.com/terraform/tutorials/provision/packer>
42. Immutable Infrastructure Using Packer, Ansible, and Terraform | by Paul Zhao - Medium, truy cập vào tháng 12 18, 2025,
<https://medium.com/paul-zhao-projects/immutable-infrastructure-using-packer-ansible-and-terraform-a275aa6e9ff7>
43. Auto Scaling Group with Rolling Deployment Module - Gruntwork Docs, truy cập vào tháng 12 18, 2025,
<https://docs.gruntwork.io/reference/modules/terraform-aws-asg/asg-rolling-deployment/>
44. Updating AWS Autoscaling Group : r/Terraform - Reddit, truy cập vào tháng 12 18, 2025,
https://www.reddit.com/r/Terraform/comments/1b3t37s/updating_aws_autoscaling_group/
45. From Code to Cloud: Automating EKS Deployment with GitHub Actions and Terraform CI/CD Pipeline, truy cập vào tháng 12 18, 2025,
<https://shriharidas73.medium.com/from-code-to-cloud-automating-eks-deployment-with-github-actions-and-terraform-ci-cd-pipeline-fbaaf691efd1>
46. Building a CI/CD Pipeline for Terraform with GitHub Actions (Step-by-Step Guide), truy cập vào tháng 12 18, 2025,
<https://dev.to/terrateam/building-a-cicd-pipeline-for-terraform-with-github-actions-step-by-step-guide-3dlp>
47. dflook/terraform-github-actions: GitHub actions for Terraform and OpenTofu, truy cập vào tháng 12 18, 2025,
<https://github.com/dflook/terraform-github-actions>
48. Implementing Terraform drift detection with GitHub Actions - Terrateam, truy cập vào tháng 12 18, 2025,
<https://terrateam.io/blog/terraform-drift-detection-github-actions>
49. Automate AWS deployments with HCP Terraform and GitHub Actions - HashiCorp, truy cập vào tháng 12 18, 2025,
<https://www.hashicorp.com/en/blog/automate-aws-deployments-with-hcp-terraform-and-github-actions>
50. Implementing Continuous Drift Detection in CI/CD Pipelines with GitHub Actions Workflow, truy cập vào tháng 12 18, 2025,
<https://www.firefly.ai/academy/implementing-continuous-drift-detection-in-ci-cd-pipelines-with-github-actions-workflow>
51. AWS Blue/Green Deployment Using Terraform Guide | by Nitin Yadav | Medium, truy cập vào tháng 12 18, 2025,
<https://medium.com/@nitinyadav745/aws-blue-green-deployment-using-terraform-guide-86131362ee67>
52. Deployment strategies - Introduction to DevOps on AWS, truy cập vào tháng 12 18, 2025,

<https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/deployment-strategies.html>

53. Rolling deployment with Terraform on AWS | by Prashant Kalkar - Medium, truy cập vào tháng 12 18, 2025,

<https://medium.com/inspiredbrilliance/rolling-deployment-with-terraform-on-aws-6a0d1587c82c>